Applied Economics

Plotting Data

Pello Aspuru

Winter 2026/2027

Introduction

Previous sessions:

- Set up R
- Download, scrape and merge datasets.

Today: Visualize data with **graphs**.

- Review types of variables, how they match to graph types.
- Learn how to plot data with LLM-s and `ggplot2`.
- **Discuss** what makes a good graph and how to improve them.

Data Plots and Econometric Analysis

Advantages	Disadvantages
Easy to read	Misleading: axis scales, color use...
Shows variables' distribution: outliers, skewness...	Omitted variables: Only shows 2–3 at once.
Fewer assumptions: no functional form choice.	No causal effect, no significance test.

Plots and econometric analysis are **complements**:

1. Plots help to understand variables' features and correlates.
2. Econometric analysis to establish causal relationships and test their significance.

The author holds significant **discretion**: be honest with yourself and your audience.

- Decide the purpose of the graph.
- Tradeoffs: what to show, how to show it, and what to skip.

Data Plots and Econometric Analysis

Advantages	Disadvantages
Easy to read	Misleading: axis scales, color use...
Shows variables' distribution: outliers, skewness...	Omitted variables: Only shows 2–3 at once.
Fewer assumptions: no functional form choice.	No causal effect, no significance test.

Plots and econometric analysis are **complements**:

1. Plots help to understand variables' features and correlates.
2. Econometric analysis to establish causal relationships and test their significance.

The author holds significant **discretion**: be honest with yourself and your audience.

- Decide the purpose of the graph.
- Tradeoffs: what to show, how to show it, and what to skip.

Data Plots and Econometric Analysis

Advantages	Disadvantages
Easy to read	Misleading: axis scales, color use...
Shows variables' distribution: outliers, skewness...	Omitted variables: Only shows 2–3 at once.
Fewer assumptions: no functional form choice.	No causal effect, no significance test.

Plots and econometric analysis are **complements**:

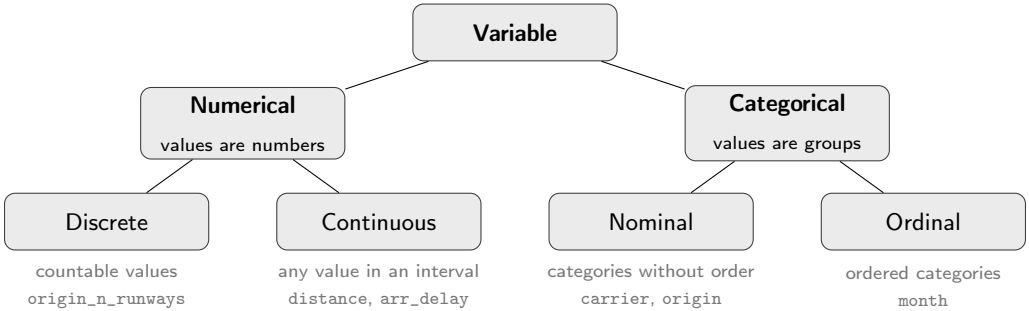
1. Plots help to understand variables' features and correlates.
2. Econometric analysis to establish causal relationships and test their significance.

The author holds significant **discretion**: be honest with yourself and your audience.

- Decide the purpose of the graph.
- Tradeoffs: what to show, how to show it, and what to skip.

Types of Variables in a Dataset

```
df_flights <- read.csv("data/df_flights_enriched.csv")
```



Types of Variables and Their Graphs

Graph	X axis Variable	Y axis Variable	Shows	ggplot2
Histogram	continuous	count per bin	distribution of a variable	`geom_histogram()`
Bar chart	categorical	count per category	number of obs. per category	`geom_bar()`
Box plot	categorical	continuous	distribution within category	`geom_boxplot()`
Scatter plot	continuous	continuous	correlation of two variables	`geom_point()`
Line graph	time	continuous	evolution of a variable	`geom_line()`

Readability and Information Trade-off

- Colour categories, patterns, or shapes can show a **additional categorical variables**.
- More information can make the graph harder to read.

Types of Variables and Their Graphs

Graph	X axis Variable	Y axis Variable	Shows	ggplot2
Histogram	continuous	count per bin	distribution of a variable	`geom_histogram()`
Bar chart	categorical	count per category	number of obs. per category	`geom_bar()`
Box plot	categorical	continuous	distribution within category	`geom_boxplot()`
Scatter plot	continuous	continuous	correlation of two variables	`geom_point()`
Line graph	time	continuous	evolution of a variable	`geom_line()`

Readability and Information Trade-off

- Colour categories, patterns, or shapes can show a **additional categorical variables**.
- More information can make the graph harder to read.

Plotting Data with ggplot2

ggplot2 is the most widely used R package for graphs. A plot is built in **layers** joined with + sign:

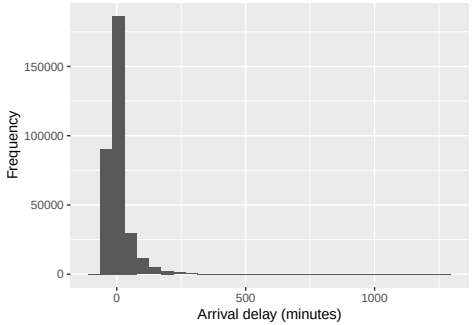
1. Data layer: `ggplot(data, aes(...))`: identifies the dataset and its variables.
2. Graph layer: `geom_...()`: sets the type of graph (histogram, points, lines...).
3. Appearance layers: `labs()`, `theme()`, ...: labels and appearance.

Manually Coding graphs

- Coding ggplot2 graphs manually is tedious and iterative.
- Any minor mistake leads to an error message.

Example: coding a histogram of arrival delays.

```
# Histogram of arrival delays
ggplot(df_flights, aes(x = arr_delay)) +
  geom_histogram(bins = 30) +
  labs(x = "Arrival delay (minutes)",
       y = "Frequency")
```



Vibecoding Graphs

LLM Prompt → LLM translates prompt into an R script → New graph.

Avoid graph **congestion**: what is the purpose of the graph?

- The programming language, the package.
- The dataset and variable names.
- The type of graph.

Prompt Example

I am working on R using the `ggplot2` graphs package. I have a continuous variable `df_flights$arr_delay`, representing the arrival delay of flights that departed from NYC airports. The dataset is based on the "flights" dataset from the `nycflights13` R package. Write an R script using ggplot2 that shows a histogram of this variable.

Vibecoding Graphs

LLM Prompt → LLM translates prompt into an R script → New graph.

Avoid graph **congestion**: what is the purpose of the graph?

- The programming language, the package.
- The dataset and variable names.
- The type of graph.

Prompt Example

I am working on R using the `ggplot2` graphs package. I have a continuous variable `df_flights$arr_delay`, representing the arrival delay of flights that departed from NYC airports. The dataset is based on the "flights" dataset from the `nycflights13` R package. Write an R script using ggplot2 that shows a histogram of this variable.

Where We Are

So far

- Graphs and econometric analysis are **complements**, and the author has **discretion**.
- The **type of variable** (continuous, categorical, time) determines the type of graph.
- `ggplot2` builds graphs in **layers**: data, graph type, appearance.
- Manual coding is tedious → **vibecoding**: a precise prompt gets an LLM to write the script.

Next: vibecode the five basic graphs

- **Histogram**, bar chart, box plot, scatter plot, line graph.
- For each: what it shows → prompt → code → graph.
- Then **discuss and improve** the first attempt with follow-up prompts.

Where We Are

So far

- Graphs and econometric analysis are **complements**, and the author has **discretion**.
- The **type of variable** (continuous, categorical, time) determines the type of graph.
- `ggplot2` builds graphs in **layers**: data, graph type, appearance.
- Manual coding is tedious → **vibecoding**: a precise prompt gets an LLM to write the script.

Next: vibecode the five basic graphs

- **Histogram**, bar chart, box plot, scatter plot, line graph.
- For each: what it shows → prompt → code → graph.
- Then **discuss and improve** the first attempt with follow-up prompts.

Histogram

Represents the distribution of a **single continuous variable**.

- **Horizontal axis**: the values of the variable, divided into intervals of equal width (**bins**).
- **Vertical axis**: the **number of observations** (frequency) whose value falls in each bin.
- The **height** of each bar is the count.

I am working on R using the `ggplot2` graphs package. I have a continuous variable `df_flights$arr_delay`, representing the arrival delay of flights that departed from NYC airports. The dataset is based on the "flights" dataset from the `nycflights13` R package. Write an R script using ggplot2 that shows a histogram of this variable.

Histogram

Represents the distribution of a **single continuous variable**.

- **Horizontal axis**: the values of the variable, divided into intervals of equal width (**bins**).
- **Vertical axis**: the **number of observations** (frequency) whose value falls in each bin.
- The **height** of each bar is the count.

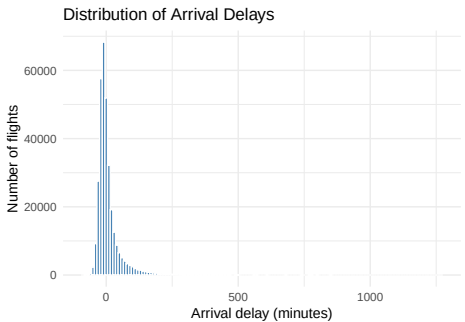
```
library(ggplot2)

ggplot(df_flights, aes(x = arr_delay)) +
  geom_histogram(binwidth = 10,
                 fill = "steelblue",
                 color = "white",
                 na.rm = TRUE) +
  labs(title = "Distribution of Arrival Delays",
       x = "Arrival delay (minutes)",
       y = "Number of flights") +
  theme_minimal()
```

Histogram

Represents the distribution of a **single continuous variable**.

- **Horizontal axis**: the values of the variable, divided into intervals of equal width (**bins**).
- **Vertical axis**: the **number of observations** (frequency) whose value falls in each bin.
- The **height** of each bar is the count.



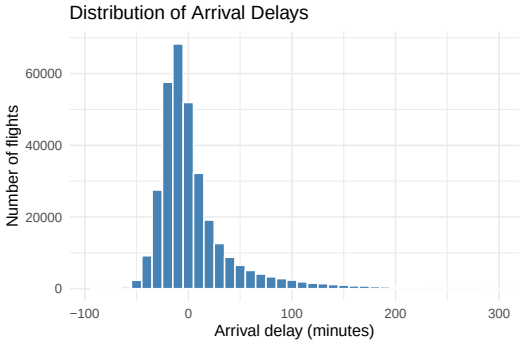
How would you improve this histogram?

Refining the Histogram

I want to improve the histogram of arrival delays. I want to reduce the horizontal axis range to a maximum of 300 minutes. Add a footnote that explains this axis range limitation.

```
n_above <- sum(df_flights$arr_delay > 300,
               na.rm = TRUE)

ggplot(df_flights, aes(x = arr_delay)) +
  geom_histogram(binwidth = 10,
                 fill = "steelblue",
                 color = "white",
                 na.rm = TRUE) +
  coord_cartesian(xlim = c(NA, 300)) +
  labs(title = "Distribution of Arrival Delays",
       x = "Arrival delay (minutes)",
       y = "Number of flights",
       caption = paste0(
         "Note: horizontal axis limited to 300 ",
         "minutes. ", n_above, " flights with ",
         "longer delays are not shown.")) +
  theme_minimal()
```



Note: horizontal axis limited to 300 minutes. 611 flights with longer delays are not shown.

Bar Chart

Represents the frequency of a **single categorical variable**.

- **Horizontal axis**: the categories of the variable (months of the year), one bar per category.
- **Vertical axis**: the **number of observations**.
- Bar **height** is observation count.

I am working on R using the `ggplot2` graphs package. I have a categorical variable `df_flights$month`, representing the months of the year for each flight. Write an R script using ggplot2 that shows a bar chart of this variable.

Bar Chart

Represents the frequency of a **single categorical variable**.

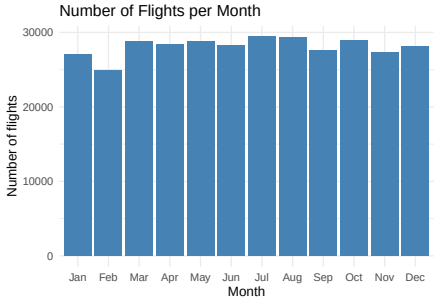
- **Horizontal axis**: the categories of the variable (months of the year), one bar per category.
- **Vertical axis**: the **number of observations**.
- Bar **height** is observation count.

```
ggplot(df_flights,
       aes(x = factor(month,
                      levels = 1:12,
                      labels = month.abb))) +
  geom_bar(fill = "steelblue") +
  labs(title = "Number of Flights per Month",
       x = "Month",
       y = "Number of flights") +
  theme_minimal()
```

Bar Chart

Represents the frequency of a **single categorical variable**.

- **Horizontal axis**: the categories of the variable (months of the year), one bar per category.
- **Vertical axis**: the **number of observations**.
- Bar **height** is observation count.



Bar Chart: Stacked Colours

A **second categorical variable** can be shown with the **colour** of the bars.

- One colour per airport, shown in the **legend**.
- Each colour counts the flights in a month **and** from one airport.
- **Stacked** (default): total bar height is still the monthly total.

Can you improve the previous bar chart by classifying the flights by their origin airport? Each airport should be represented by a different colour, with the bars stacked on top of each other. Please add a legend to the graph.

Bar Chart: Stacked Colours

A **second categorical variable** can be shown with the **colour** of the bars.

- One colour per airport, shown in the **legend**.
- Each colour counts the flights in a month **and** from one airport.
- **Stacked** (default): total bar height is still the monthly total.

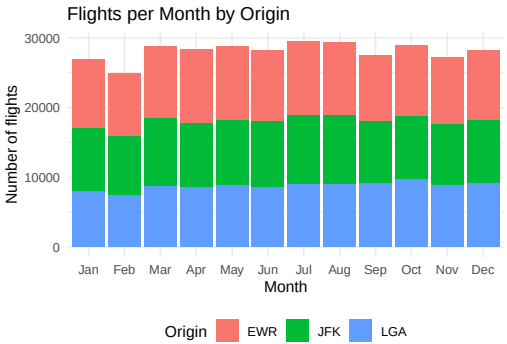
```
library(ggplot2)

ggplot(df_flights,
       aes(x = factor(month,
                      levels = 1:12,
                      labels = month.abb),
           fill = origin)) +
  geom_bar() +  # stacked by default
  labs(title = "Flights per Month by Origin",
       x = "Month",
       y = "Number of flights",
       fill = "Origin") +
  theme_minimal() +
  theme(legend.position = "bottom")
```

Bar Chart: Stacked Colours

A **second categorical variable** can be shown with the **colour** of the bars.

- One colour per airport, shown in the **legend**.
- Each colour counts the flights in a month **and** from one airport.
- **Stacked** (default): total bar height is still the monthly total.



Bar Chart: Side-by-Side Colours

Alternatively, the coloured bars can be placed **side by side**.

- One colour per airport, shown in the **legend**.
- Each colour counts the flights in a month **and** from one airport.
- One bar per airport within each month.
- Set with `position = "dodge"`.

Can you change the previous bar chart so that the bars of each airport are placed side by side instead of stacked?

Bar Chart: Side-by-Side Colours

Alternatively, the coloured bars can be placed **side by side**.

- One colour per airport, shown in the **legend**.
- Each colour counts the flights in a month **and** from one airport.
- One bar per airport within each month.
- Set with `position = "dodge"`.

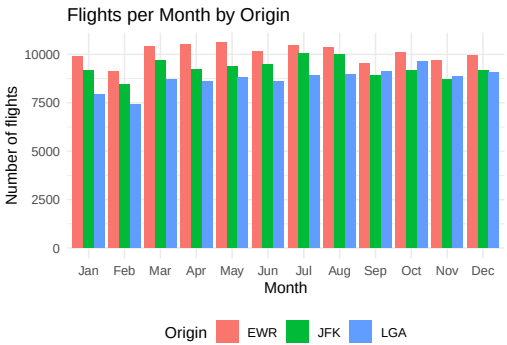
```
library(ggplot2)

ggplot(df_flights,
       aes(x = factor(month,
                      levels = 1:12,
                      labels = month.abb),
           fill = origin)) +
  geom_bar(position = "dodge") +  # side by side
  labs(title = "Flights per Month by Origin",
       x = "Month",
       y = "Number of flights",
       fill = "Origin") +
  theme_minimal() +
  theme(legend.position = "bottom")
```

Bar Chart: Side-by-Side Colours

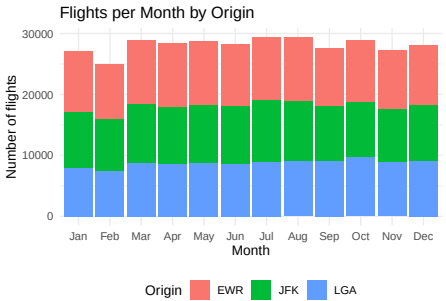
Alternatively, the coloured bars can be placed **side by side**.

- One colour per airport, shown in the **legend**.
- Each colour counts the flights in a month **and** from one airport.
- One bar per airport within each month.
- Set with `position = "dodge"`.

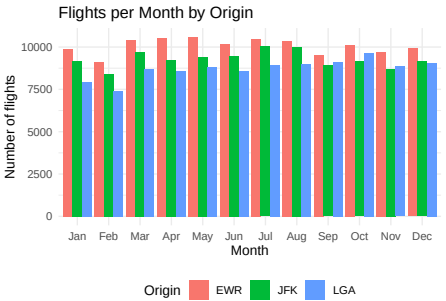


Bar Chart Type Discussion

Stacked (default)



Side by side (`position = "dodge"`)



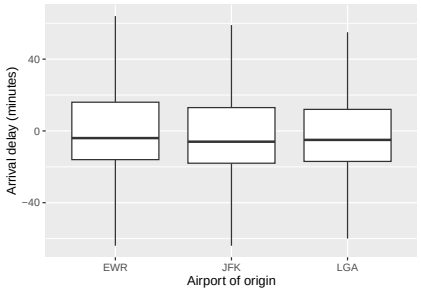
Which do you prefer? Why?

Box Plot

Summary statistics of a **continuous variable** (delay time) within each **category** (airport of origin).

- **Horizontal axis**: the categories of the variable.
- **Vertical axis**: continuous variable.
- The **line** inside the box is the median.
- The **box** ranges from the 25th to the 75th percentile.
- The **whiskers** extend to the most extreme values that lie within 1.5 box heights.
- **Points** outside the whiskers.

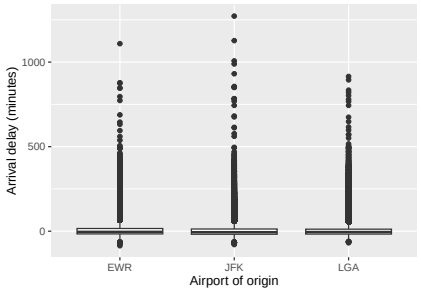
Decision: should we include outliers?



Box Plot

Summary statistics of a **continuous variable** (delay time) within each **category** (airport of origin).

- **Horizontal axis**: the categories of the variable.
- **Vertical axis**: continuous variable.
- The **line** inside the box is the median.
- The **box** ranges from the 25th to the 75th percentile.
- The **whiskers** extend to the most extreme values that lie within 1.5 box heights.
- **Points** outside the whiskers.

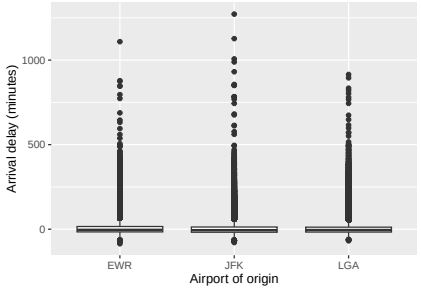


Decision: should we include outliers?

Box Plot

Summary statistics of a **continuous variable** (delay time) within each **category** (airport of origin).

- **Horizontal axis**: the categories of the variable.
- **Vertical axis**: continuous variable.
- The **line** inside the box is the median.
- The **box** ranges from the 25th to the 75th percentile.
- The **whiskers** extend to the most extreme values that lie within 1.5 box heights.
- **Points** outside the whiskers.

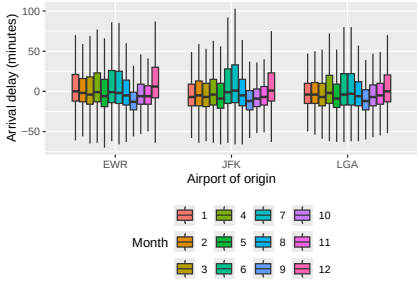


Decision: should we include outliers?

Box Plot: Adding Categories with Colours

A **second categorical variable** (month of the year) can be shown with the **colour** of the boxes.

- One colour per month of the year, shown in the **legend**.
- Within each airport, the boxes of each month are placed **side by side**.
- Each box summarises the delays of the flights from one airport **and** in one month.
- With 12 categories, that is 36 boxes: hard to read and to compare.

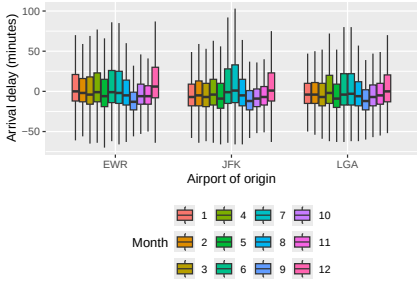


What does the graph show? How can we improve it?

Box Plot: Adding Categories with Colours

A **second categorical variable** (month of the year) can be shown with the **colour** of the boxes.

- One colour per month of the year, shown in the **legend**.
- Within each airport, the boxes of each month are placed **side by side**.
- Each box summarises the delays of the flights from one airport **and** in one month.
- With 12 categories, that is 36 boxes: hard to read and to compare.



What does the graph show? How can we improve it?

Scatter Plot

Represents the correlation between **two continuous variables** (distance and arrival delay).

- **Horizontal axis**: the first continuous variable (distance in miles).
- **Vertical axis**: the second continuous variable (arrival delay in minutes).
- Each **point** is one observation (one flight), placed at its distance and its delay.
- With many observations, points overlap: transparency (`alpha = 0.2`) shows where they concentrate.

I am working on R using the `ggplot2` graphs package. I have two continuous variables, `df_flights$distance` and `df_flights$arr_delay`, representing the distance (in miles) and the arrival delay (in minutes) of each flight. Write an R script using ggplot2 that shows a scatter plot of arrival delay against distance.

Scatter Plot

Represents the correlation between **two continuous variables** (distance and arrival delay).

- **Horizontal axis**: the first continuous variable (distance in miles).
- **Vertical axis**: the second continuous variable (arrival delay in minutes).
- Each **point** is one observation (one flight), placed at its distance and its delay.
- With many observations, points overlap: transparency (`alpha = 0.2`) shows where they concentrate.

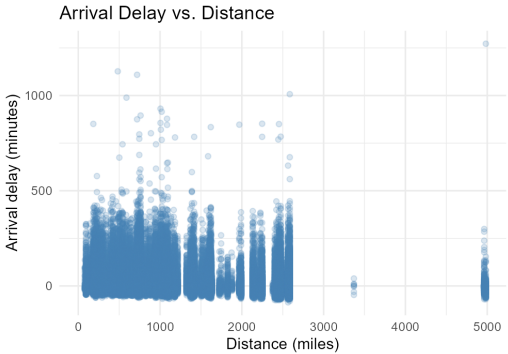
```
library(ggplot2)

ggplot(df_flights,
       aes(x = distance,
           y = arr_delay)) +
  geom_point(alpha = 0.2,  # many points
             color = "steelblue",
             na.rm = TRUE) +
  labs(title = "Arrival Delay vs. Distance",
       x = "Distance (miles)",
       y = "Arrival delay (minutes)") +
  theme_minimal()
```

Scatter Plot

Represents the correlation between **two continuous variables** (distance and arrival delay).

- **Horizontal axis**: the first continuous variable (distance in miles).
- **Vertical axis**: the second continuous variable (arrival delay in minutes).
- Each **point** is one observation (one flight), placed at its distance and its delay.
- With many observations, points overlap: transparency (`alpha = 0.2`) shows where they concentrate.



What does the plot show?

Binscatter Plot

When scatterplots are too crowded → binscatter plot.

Each point is the average of x and y within one bin of x.

- **Horizontal axis**: the first variable (distance), divided into 20 **bins** with the same number of flights each.
- **Vertical axis**: the **mean** of the second variable (arrival delay) within each bin.
- Each **point** is one bin, not one flight.

I have this scatterplot that is too crowded [paste the previous code here]. I want to turn it into a binscatter plot with 20 bins. Please reflect any other arbitrary decisions in the graph footnote.

Binscatter Plot

When scatterplots are too crowded → binscatter plot.

Each point is the average of x and y within one bin of x.

- **Horizontal axis**: the first variable (distance), divided into 20 **bins** with the same number of flights each.
- **Vertical axis**: the **mean** of the second variable (arrival delay) within each bin.
- Each **point** is one bin, not one flight.

```
# 20 bins of distance, equal number of flights
df_bins <- df_flights |>
  filter(!is.na(arr_delay)) |>
  mutate(bin = ntile(distance, 20)) |>
  group_by(bin) |>
  summarise(distance  = mean(distance),
            arr_delay = mean(arr_delay))
```

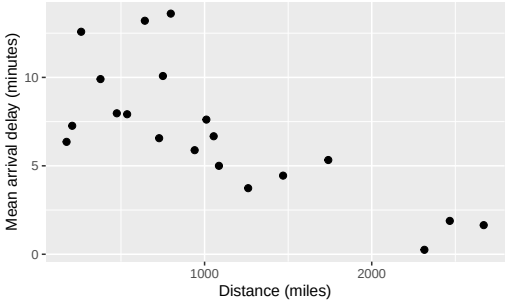
```
ggplot(df_bins, aes(x = distance,
                    y = arr_delay)) +
  geom_point(size = 2) +
  labs(x = "Distance (miles)",
       y = "Mean arrival delay (minutes)",
       caption = paste(
         "Note: 20 distance bins with equal",
         "numbers of flights. Points are bin",
         "means. Cancelled flights excluded.",
         sep = "\n"))
```

Binscatter Plot

When scatterplots are too crowded → binscatter plot.

Each point is the average of x and y within one bin of x.

- **Horizontal axis**: the first variable (distance), divided into 20 **bins** with the same number of flights each.
- **Vertical axis**: the **mean** of the second variable (arrival delay) within each bin.
- Each **point** is one bin, not one flight.



Note: 20 distance bins with equal numbers of flights. Points are bin means. Cancelled flights excluded.

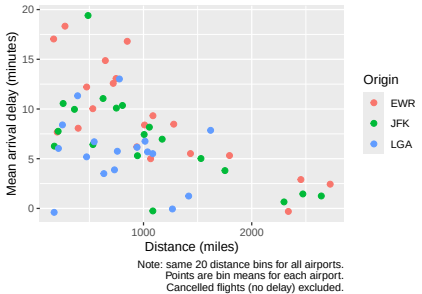
The hidden pattern **emerges**: longer flights arrive, on average, with shorter delays.

Binscatter improvements: Adding Categories with Colours

Can you improve the previous binscatter by classifying the flights by their origin airport? Each airport should be represented by a different colour. Please add a legend to the graph.

```
# Same 20 bins, means by bin and airport
df_bins <- df_flights |>
  filter(!is.na(arr_delay)) |>
  mutate(bin = ntile(distance, 20)) |>
  group_by(bin, origin) |>
  summarise(distance  = mean(distance),
            arr_delay = mean(arr_delay))

ggplot(df_bins, aes(x = distance,
                    y = arr_delay,
                    colour = origin)) +
  geom_point(size = 2) +
  labs(x = "Distance (miles)",
       y = "Mean arrival delay (minutes)",
       colour = "Origin",
       caption = paste(
         "Note: same 20 distance bins for all airports.",
         "Points are bin means for each airport.",
         "Cancelled flights (no delay) excluded.",
         sep = "\n"))
```

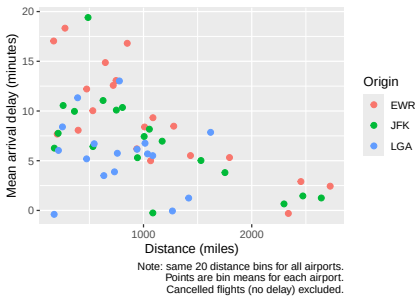


Binscatter improvements: Adding Categories with Colours

Can you improve the previous binscatter by classifying the flights by their origin airport? Each airport should be represented by a different colour. Please add a legend to the graph.

```
# Same 20 bins, means by bin and airport
df_bins <- df_flights |>
  filter(!is.na(arr_delay)) |>
  mutate(bin = ntile(distance, 20)) |>
  group_by(bin, origin) |>
  summarise(distance  = mean(distance),
            arr_delay = mean(arr_delay))

ggplot(df_bins, aes(x = distance,
                    y = arr_delay,
                    colour = origin)) +
  geom_point(size = 2) +
  labs(x = "Distance (miles)",
       y = "Mean arrival delay (minutes)",
       colour = "Origin",
       caption = paste(
         "Note: same 20 distance bins for all airports.",
         "Points are bin means for each airport.",
         "Cancelled flights (no delay) excluded.",
         sep = "\n"))
```



Line Graph

A line graph shows how **one continuous variable evolves over time**.

- **Horizontal axis**: time (here, each day of 2013).
- **Vertical axis**: the value of the variable at each date (here, the number of flights that day).
- Points are **connected in time order**.

I am working on R using the `ggplot2` graphs package. Each row of `df_flights` is a flight that departed from NYC in 2013, with its date in the variables `year`, `month` and `day`. Write an R script using ggplot2 that shows a line graph of the number of flights per day.

Line Graph

A line graph shows how **one continuous variable evolves over time**.

- **Horizontal axis**: time (here, each day of 2013).
- **Vertical axis**: the value of the variable at each date (here, the number of flights that day).
- Points are **connected in time order**.

```
library(ggplot2)
library(dplyr)

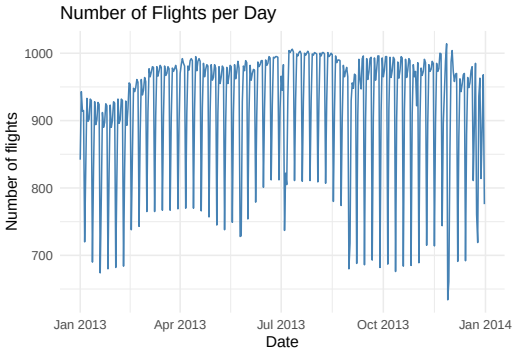
# Count the flights of each day
daily <- df_flights |>
  mutate(date = as.Date(
    paste(year, month, day, sep = "-"))) |>
  count(date)

ggplot(daily, aes(x = date, y = n)) +
  geom_line(color = "steelblue") +
  labs(title = "Number of Flights per Day",
       x = "Date",
       y = "Number of flights") +
  theme_minimal()
```

Line Graph

A line graph shows how **one continuous variable evolves over time**.

- **Horizontal axis**: time (here, each day of 2013).
- **Vertical axis**: the value of the variable at each date (here, the number of flights that day).
- Points are **connected in time order**.



What does the graph show? How can we improve it?

Line Graph: Aggregating to Months

The **time unit** we choose changes what the graph shows.

- Daily counts are **noisy**: weekdays and weekends alternate.
- Aggregating to **months** smooths the noise and shows the seasonal pattern.
- Decision: the time unit (day, week, month) is the author's choice.

Can you change the previous line graph so that it shows the number of flights per month instead of per day? Label each month on the horizontal axis.

Line Graph: Aggregating to Months

The **time unit** we choose changes what the graph shows.

- Daily counts are **noisy**: weekdays and weekends alternate.
- Aggregating to **months** smooths the noise and shows the seasonal pattern.
- Decision: the time unit (day, week, month) is the author's choice.

```
library(ggplot2)
library(dplyr)

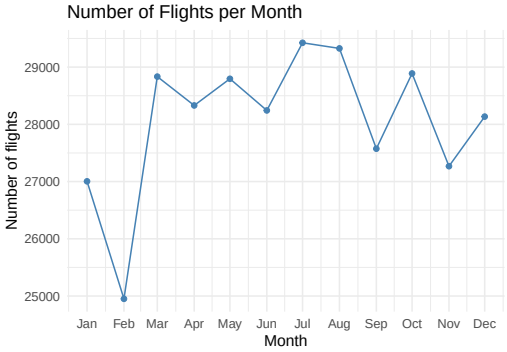
# Count the flights of each month
monthly <- df_flights |>
  mutate(ym = as.Date(
    paste(year, month, 1, sep = "-"))) |>
  count(ym)

ggplot(monthly, aes(x = ym, y = n)) +
  geom_line(color = "steelblue") +
  geom_point(color = "steelblue") +
  scale_x_date(date_breaks = "1 month",
               date_labels = "%b") +
  labs(title = "Number of Flights per Month",
       x = "Month",
       y = "Number of flights") +
  theme_minimal()
```

Line Graph: Aggregating to Months

The **time unit** we choose changes what the graph shows.

- Daily counts are **noisy**: weekdays and weekends alternate.
- Aggregating to **months** smooths the noise and shows the seasonal pattern.
- Decision: the time unit (day, week, month) is the author's choice.



What is happening in February?

Line Graph: Starting the Vertical Axis at Zero

Where the vertical axis starts changes how **large** the changes look.

- By default, `ggplot2` **zooms** the vertical axis on the range of the data.
- Small changes then look **dramatic**.
- Starting at **zero** shows their true size: every month has roughly 25,000 to 29,500 flights.

Can you make the vertical axis of the previous graph start at zero?

Line Graph: Starting the Vertical Axis at Zero

Where the vertical axis starts changes how **large** the changes look.

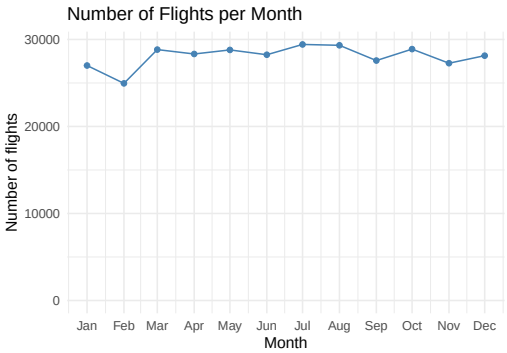
- By default, ggplot2 **zooms** the vertical axis on the range of the data.
- Small changes then look **dramatic**.
- Starting at **zero** shows their true size: every month has roughly 25,000 to 29,500 flights.

```
# Same monthly data as before
ggplot(monthly, aes(x = ym, y = n)) +
  geom_line(color = "steelblue") +
  geom_point(color = "steelblue") +
  expand_limits(y = 0) +  # axis from zero
  scale_x_date(date_breaks = "1 month",
               date_labels = "%b") +
  labs(title = "Number of Flights per Month",
       x = "Month",
       y = "Number of flights") +
  theme_minimal()
```

Line Graph: Starting the Vertical Axis at Zero

Where the vertical axis starts changes how **large** the changes look.

- By default, `ggplot2` **zooms** the vertical axis on the range of the data.
- Small changes then look **dramatic**.
- Starting at **zero** shows their true size: every month has roughly 25,000 to 29,500 flights.



Line Graph: Adding Categories with Colours

A **categorical variable** can be shown with one line per category.

- One colour per airport, shown in the **legend**.
- Each point counts the flights in a month **and** from one airport.
- Lines make it easy to **compare trends** across airports.

Can you improve the previous graph by showing one line per origin airport? Each airport should be represented by a different colour. Please add a legend to the graph.

Line Graph: Adding Categories with Colours

A **categorical variable** can be shown with one line per category.

- One colour per airport, shown in the **legend**.
- Each point counts the flights in a month **and** from one airport.
- Lines make it easy to **compare trends** across airports.

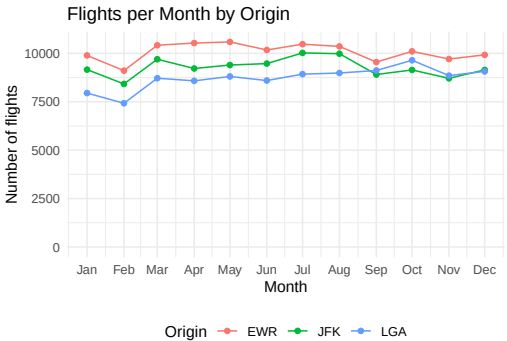
```
# Count the flights of each month and airport
monthly_origin <- df_flights |>
  mutate(ym = as.Date(
    paste(year, month, 1, sep = "-"))) |>
  count(ym, origin)
```

```
ggplot(monthly_origin, aes(x = ym, y = n,
                           colour = origin)) +
  geom_line() +
  geom_point() +
  expand_limits(y = 0) +
  scale_x_date(date_breaks = "1 month",
               date_labels = "%b") +
  labs(title = "Flights per Month by Origin",
       x = "Month", y = "Number of flights",
       colour = "Origin") +
  theme_minimal() +
  theme(legend.position = "bottom")
```

Line Graph: Adding Categories with Colours

A **categorical variable** can be shown with one line per category.

- One colour per airport, shown in the **legend**.
- Each point counts the flights in a month **and** from one airport.
- Lines make it easy to **compare trends** across airports.



Line Graph: Comparisons in Percentages

Airports of different sizes can be compared with an **index**.

- Raw counts differ in **size** across airports: their changes are hard to compare.
- Each airport's first full week is set to **100**; the graph shows the **% change** from it.
- All lines start at 0%: they show **relative** changes.
- Decisions: the baseline period and the time unit (weeks).

I want to represent how the number of flights in each airport has changed over the months. For that, I want to aggregate the data to weekly level, then determine the first period as level 100 for each airport, and finally plot the percentage change of flights with respect to the baseline.

Line Graph: Comparisons in Percentages

Airports of different sizes can be compared with an **index**.

- Raw counts differ in **size** across airports: their changes are hard to compare.
- Each airport's first full week is set to **100**; the graph shows the **% change** from it.
- All lines start at 0%: they show **relative** changes.
- Decisions: the baseline period and the time unit (weeks).

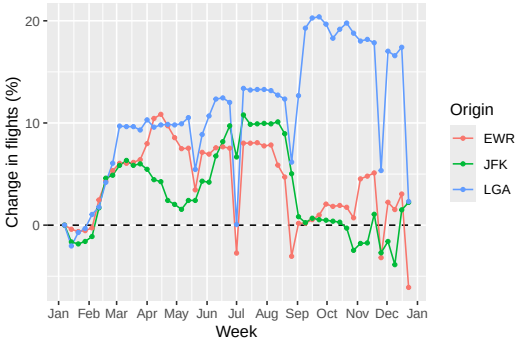
```
weekly <- df_flights |>
  # week = Monday that starts each week
  mutate(date = as.Date(paste(year, month, day,
                              sep = "-")),
         week = as.Date(cut(date, "week"))) |>
  group_by(origin, week) |>
  summarise(n = n(), days = n_distinct(date),
            .groups = "drop") |>
  # drop incomplete weeks (start/end of year)
  filter(days == 7) |>
  group_by(origin) |>
  mutate(index = 100 * n / first(n),
         pct_change = index - 100) |>
  ungroup()

ggplot(weekly, aes(x = week, y = pct_change,
                   colour = origin)) +
  geom_hline(yintercept = 0, linetype = "dashed") +
  geom_line() + geom_point(size = 1) +
  scale_x_date(date_breaks = "1 month",
               date_labels = "%b") +
  labs(x = "Week", y = "Change in flights (%)",
       colour = "Origin",
       caption = paste("Note: first full week of 2013 = 100.",
                       "Incomplete weeks excluded."))
```

Line Graph: Comparisons in Percentages

Airports of different sizes can be compared with an **index**.

- Raw counts differ in **size** across airports: their changes are hard to compare.
- Each airport's first full week is set to **100**; the graph shows the **% change** from it.
- All lines start at 0%: they show **relative** changes.
- Decisions: the baseline period and the time unit (weeks).



Note: first full week of 2013 = 100. Incomplete weeks excluded.

Graphs: Summary

Graphs and econometric models

+ Easy to read, show whole distributions, no functional form.
– Only 2–3 variables at once, no causal effect, no significance test.

Graphs for **preliminary** analysis.

- Plot before you estimate: distributions, outliers, missing values, patterns.
- Choose graph type wisely: what is the purpose of the visualization?

Vibecoding graphs in R `ggplot2` package.

- A precise prompt: language and package, dataset and variable names, graph type.
- **Read** the code, check the graph, and **iterate** with follow-up prompts.

Graphs: Summary

Graphs and econometric models

+ Easy to read, show whole distributions, no functional form.
– Only 2–3 variables at once, no causal effect, no significance test.

Graphs for **preliminary** analysis.

- Plot before you estimate: distributions, outliers, missing values, patterns.
- Choose graph type wisely: what is the purpose of the visualization?

Vibecoding graphs in R `ggplot2` package.

- A precise prompt: language and package, dataset and variable names, graph type.
- **Read** the code, check the graph, and **iterate** with follow-up prompts.

Graphs: Summary

Graphs and econometric models

+ Easy to read, show whole distributions, no functional form.

– Only 2–3 variables at once, no causal effect, no significance test.

Graphs for **preliminary** analysis.

- Plot before you estimate: distributions, outliers, missing values, patterns.
- Choose graph type wisely: what is the purpose of the visualization?

Vibecoding graphs in R `ggplot2` package.

- A precise prompt: language and package, dataset and variable names, graph type.
- **Read** the code, check the graph, and **iterate** with follow-up prompts.